

Appunti di Algoritmi Distribuiti e Reti Complesse

Zbirciog Ionut Georgian

8 marzo 2026

Indice

1	Modellazione Del Sistema	2
1.1	Definizioni Elementari	2
1.2	Assiomi e Restrizioni	5
1.2.1	Assiomi	5
1.2.2	Resitrizioni	6
2	Broadcast	8
2.1	Algoritmo Flooding	9

Capitolo 1

Modellazione Del Sistema

Definizione 1.0.1 (Sistema Distribuito). Un *sistema distribuito* è un insieme **entità** (dette anche **nodi**, **agenti**) che eseguono un algoritmo cooperando tra loro, comunicando e coordinandosi per raggiungere un obiettivo comune. Ogni nodo ha una propria memoria locale e può comunicare con altri nodi attraverso un canale di comunicazione. Internamente, ciascun nodo possiede un clock locale, che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse. Inoltre, i nodi possono subire guasti, come crash o malfunzionamenti, che possono influenzare il comportamento del sistema.

Tale ambiente distribuito viene modellato mediante grafo $G = (V, E)$, dove V ($|V| = n$) è l'insieme dei vertici (nodi) e E ($|E| = m$) è l'insieme degli archi (interconnessioni). In particolar modo, data una rete costituita da agenti e interconnessioni tra di essi, il grafo sottostante ha un vertice per ciascun agente del sistema e un arco tra due nodi per ogni interconnessione tra i corrispettivi agenti.

1.1 Definizioni Elementari

Definizione 1.1.1 (Entità). Un'ambiente distribuito è costituito da un insieme $\mathcal{E}$ di entità. Ogni nodo $x \in \mathcal{E}$ della rete, ha delle capacità computazionali, le quali sono definite come un insieme di operazioni possibili, tra le quali

- **Comunicazione:** capacità di inviare e ricevere messaggi ad altri nodi della rete.
- **Local storage & processing:** capacità di memorizzare e elaborare informazioni localmente.
- **Clock:** capacità di tenere traccia del tempo, impostando un clock locale che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse.
- **Modificare registri locali:** capacità di modificare i propri **registri locali**, che rappresentano lo stato interno del nodo.

Definizione 1.1.2 (Stato). Definiamo Σ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$. L'insieme degli stati è definito a priori e deve essere un insieme finito. Ad

esempio,

$$\Sigma = \{\text{idle}, \text{computing}, \text{waiting}, \dots\}$$

È importante notare che ad ogni istante di tempo t , un nodo x si trova in uno stato specifico $s \in \Sigma$. Lo stato di un nodo può cambiare nel tempo a seguito di operazioni computazionali o comunicazioni con altri nodi. Non esistono stati intermedi o transitori: un nodo è sempre in uno stato ben definito, e il passaggio da uno stato all'altro avviene in modo discreto, senza stati intermedi.

Definizione 1.1.3 (Evento). In un ambiente distribuito, il comportamento di ogni entità $x \in \mathcal{E}$ è **reattivo**, ovvero è indotto a seguito del verificarsi di **eventi**. In assenza di stimoli, un'entità rimane inerte.

In generale, gli eventi possono essere di vario tipo, come ad esempio:

- **Ricezione di un messaggio**: quando un nodo riceve un messaggio da un altro nodo, questo evento può indurre il nodo a cambiare stato o a eseguire determinate operazioni.
- **Tick del clock**: quando il clock locale di un nodo raggiunge un certo valore, questo evento può indurre il nodo a eseguire determinate operazioni o a cambiare stato.
- **Evento spontaneo**: un nodo può essere indotto a eseguire determinate operazioni o a cambiare stato in modo spontaneo.

Definizione 1.1.4 (Azione). Un' **azione** è una trasformazione che può essere eseguita da un nodo $x \in \mathcal{E}$ in risposta a un evento. Ogni azione modifica lo stato di un nodo o invia un messaggio ad un altro nodo. Formalmente, è una funzione **deterministica** che, fissato un istante di tempo t , mappa tutte le coppie (stato, evento) in un'azione specifica.

$$f : \Sigma \times \hat{E} \rightarrow \mathcal{A}$$

dove $\mathcal{A}$ è l'insieme di tutte le **sequenze di task** che possono essere eseguite, e $\hat{E}$ è l'insieme di tutti gli eventi che possono verificarsi in un ambiente distribuito.

In generale, un'azione può consistere in:

- **computare** un algoritmo;
- **inviare** messaggi ad altri nodi;
- **modificare** lo stato interno del nodo.

In questo modello, ogni azione è **atomica** e **finita**:

- **Atomica**: una volta avviata, un'azione non può essere interrotta;
- **Finita**: un'azione termina sempre in tempo finito.

Si osserva che un'entità in un certo stato potrebbe non reagire a un evento: in questo caso si assume che l'entità abbia eseguito l'azione *nil* (azione nulla).

Definizione 1.1.5 (Comportamento di un'entità). Le associazioni (stato, evento) $\rightarrow$ azione, per ogni nodo $x \in \mathcal{E}$, sono dette **regole**. Definiamo il **comportamento** di un'entità x come l'insieme di tutte le regole che governano le azioni che il nodo esegue in risposta

a eventi specifici, a seconda dello stato in cui si trova. Diremo che il comportamento di un'entità è

- **deterministico** se, per ogni coppia (stato, evento), esiste una regola che associa un'unica azione,
- **completo** se per ogni coppia (stato, evento) esiste almeno una regola che associa un'azione.

Il **comportamento** di un nodo $x \in \mathcal{E}$ è descrivibile mediante l'insieme $B(x)$ che deve essere **non-ambiguo** e **completo**, ossia:

$$\forall (s, e) \in \Sigma \times \hat{E}, \exists! a \in \mathcal{A} : f(s, e) = a$$

Definizione 1.1.6 (Comportamento del Protocollo). Definiamo il **comportamento del protocollo** come l'insieme di tutti i comportamenti dei nodi che partecipano al sistema distribuito. Formalmente è descritto come:

$$B(\mathcal{E}) = \{B(x) : x \in \mathcal{E}\}$$

Un sistema si dice **simmetrico (o omogeneo)** se tutti i nodi condividono lo stesso comportamento, ovvero:

$$B(x) \equiv B(y) \quad \forall x, y \in \mathcal{E}$$

Si nota che un sistema simmetrico è caratterizzato da un protocollo unico che viene eseguito da tutti i nodi. Questo significa che durante l'esecuzione i nodi possono effettuare azioni diverse ma il protocollo deve essere lo stesso. In un sistema asimmetrico (o eterogeneo), l'azione del singolo nodo dipende dalla sua etichetta.

In un sistema **asimmetrico (o eterogeneo)**, il comportamento del singolo nodo dipende dalla sua identificazione o dal suo ruolo. Tuttavia, qualsiasi sistema asimmetrico può essere trasformato in uno simmetrico riscrivendo l'insieme di regole in modo che l'azione eseguita dipenda dalla tipologia del nodo.

Esempio: Consideriamo un sistema con due tipologie di entità: server e client, ognuna con un insieme di regole differenti. È possibile unificare i comportamenti creando un nuovo insieme di regole simmetrico in cui ogni nodo esegue la stessa logica, ma l'azione concreta varia in base al ruolo dell'entità. In questo modo, il protocollo rimane unico e simmetrico per tutti i nodi.

Definizione 1.1.7 (Comunicazione). Le entità in un sistema distribuito interagiscono attraverso lo **scambio di messaggi insieme dei nodi con cui x può comunicare direttamente, ossia:gi (message passing)**. Un messaggio è una **sequenza finita di bit** che transita su canali di comunicazione tra nodi. La topologia di comunicazione è rappresentata da un grafo diretto $G = (V, E)$ ($V = \mathcal{E}$), dove un arco $(x, y) \in E$ indica che il nodo x può inviare messaggi al nodo y . Non tutti i nodi possono comunicare direttamente: la capacità di comunicazione tra due nodi è determinata dalla struttura della rete sottostante.

Si osserva che i nodi *non hanno una visione globale* del sistema (ossia non conoscono la topologia del grafo della rete), ma solo una **visione locale**. Formalmente, $\forall x \in V$, possiamo definire gli insiemi:

- $N_{\text{out}}(x) = \{y \in V : (x, y) \in E\} \subset V$, l'insieme dei nodi a cui x può inviare messaggi (*archi uscenti*);
- $N_{\text{in}}(x) = \{y \in V : (y, x) \in E\} \subset V$, l'insieme dei nodi da cui x può ricevere messaggi (*archi entranti*).

Dunque, definiamo il **vicinato** di un nodo x come

$$N(x) = N_{\text{out}}(x) \cup N_{\text{in}}(x)$$

1.2 Assiomi e Restrizioni

La definizione di sistema distribuito è accompagnata da 2 *assiomi fondamentali*. Qualsiasi assunzione aggiuntiva che si vuole fare su un sistema distribuito sarà chiamata *restrizione*

1.2.1 Assiomi

Finite Transmission Delays In assenza di perdite di messaggi, si assume che il tempo di trasmissione di un messaggio tra due nodi sia finito. Ciò implica che, se un nodo invia un messaggio a un altro nodo, il messaggio arriverà entro un tempo finito, anche se questo tempo può essere arbitrariamente lungo, di conseguenza, il sistema non è a conoscenza di questo tempo di trasmissione.

Formalmente, $\forall$ messaggio m inviato da un nodo x a un nodo y , esiste un tempo finito $\tau_{xy} \in \mathbb{R}^+$ tale che m arriva a y entro τ_{xy} .

Local Orientation Questo assioma stabilisce che ogni nodo distingue localmente i suoi vicini, ossia, distingue tra gli archi entranti e quelli uscenti. Un'entità è sempre in grado di inviare un messaggio solamente ad uno specifico e singolo vicino uscente. Di conseguenza, è in grado di distinguere quale dei suoi vicini gli ha inviato un messaggio. Formalmente, ogni nodo x ha una funzione locale iniettiva di orientamento, che assegna un'etichetta univoca a ciascun arco:

$$\lambda_x : E_x \rightarrow L$$

dove $E_x = \{(x, y) \in E\}$ è l'insieme degli archi incidenti su x e L è un insieme di etichette.

Possiamo quindi discutere di grafi etichettati (G, λ) ($\lambda = \{\lambda_x : x \in V\}$), dove ogni arco (**non nodo**) (x, y) ha un'etichetta **locale** $\lambda_x(x, y)$ che identifica univocamente il vicino y per il nodo x . Allo stesso modo, l'arco (y, x) avrà un'etichetta $\lambda_y(y, x)$ che identifica x per il nodo y .



Figura 1.1: Ciascun arco ha due etichette.

In generale, $\lambda_x(x, y) \neq \lambda_y(x, y)$.

1.2.2 Resitrazioni

Spesso i protocolli vengono definiti e studiati assumendo ulteriori restrizioni. Naturalmente un protocollo definito sotto un gran numero di restrizioni è generalmente debole, nel senso che, rimuovendo alcune di queste restrizioni, il protocollo potrebbe non risolvere il problema preso in analisi. Un protocollo definito su un sistema con poche restrizioni invece risulta più generale, e dunque più forte.

Si elencano alcune tipologie di restrizioni.

Message Ordering Si assume che i messaggi inviati da un nodo x a un nodo y arrivino in ordine FIFO (First In First Out) (**FIFO queue**). Ciò significa che se x invia due messaggi m_1 e m_2 a y , con m_1 inviato prima di m_2 , allora y riceverà m_1 prima di m_2 . Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce un ordine prevedibile dei messaggi tra coppie di nodi.

Bidirectional Links Si assume che le connessioni tra nodi sia *bidirezionale*, ossia, $\forall x, y \in \mathcal{E}$, se $(x, y) \in E$, allora anche $(y, x) \in E$ e inoltre $\lambda_x(x, y) = \lambda_y(y, x)$. Questa restrizione implica che se un nodo x può inviare messaggi a un nodo y , allora y può anche inviare messaggi a x , e che entrambi i nodi condividono la stessa etichetta per identificare la connessione tra di loro. Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce una comunicazione simmetrica tra i nodi. Formalmente,

$$\forall x \in V, N_{\text{out}}(x) = N_{\text{in}}(x) = N(x) \quad \text{e} \quad \lambda_x(x, y) = \lambda_y(y, x) \quad \forall y \in N_{\text{out}}(x)$$

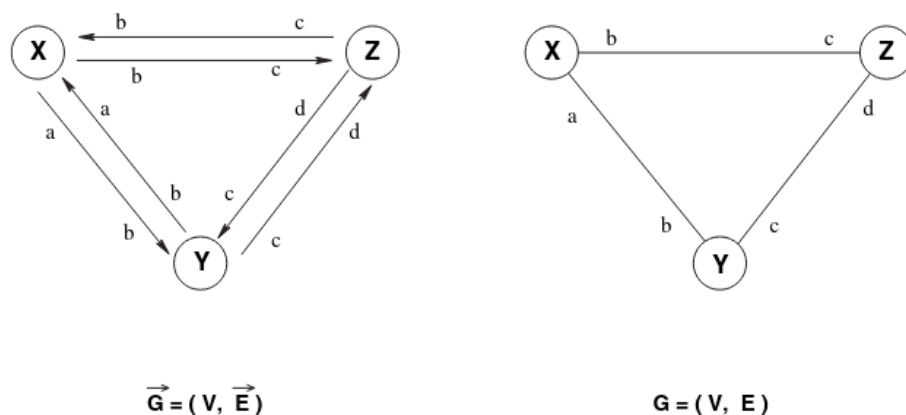


Figura 1.2: In una rete con archi bidirezionali, consideriamo il corrispondente grafo semplificato non diretto.

Reliability Restrictions Ci chiediamo: cosa succede al messaggio durante il tragitto? Possiamo fare delle assunzioni sulla disponibilità dei canali di comunicazione.

- **Guaranteed Delivery:** Ogni messaggio che viene inviato viene **ricevuto correttamente**, cioè non ci sono corruzioni di messaggi. Sotto questa restrizione, i protocolli non devono tenere conto di omissioni o corruzioni dei messaggi durante la trasmissione.

- **Partial Reliability:** Non si verificano guasti (ad esempio crash dei nodi, perdita di messaggi) dall'istante in cui si inizia ad osservare il sistema. Si osserva che, negli istanti precedenti all'inizio dell'osservazione, il sistema potrebbe aver subito guasti.
- **Total Reliability:** Non sono mai avvenuti fallimenti e mai avverranno in futuro.

Topological Restrictions In genere consistono in assunzioni sulla topologia del grafo della rete. Ad esempio assumere che il grafo sia **connesso/fortemente connesso**.

Knowledge Restrictions Assumiamo quali sono le informazioni che ogni nodo conosce riguardo la rete. Ad esempio in numero di nodi ($|V| = n$), oppure il numero di archi ($|E| = m$), il diametro della rete, l'etichettamento universale, etc... .

Time Restrictions Eccetto il tempo di trasmissione finito, il modello generale non fa assunzioni su tale quantità. Due possibili restrizioni riguardo i tempi di comunicazione sono le seguenti:

- **Bounded Communication Delay:** Esiste una costante Δ tale che, in assenza di guasti, il delay dei messaggi è al più Δ , in altre parole se un nodo x invia un messaggio m ad un nodo y , m arriverà ad y in al più Δ istanti di tempo.
- **Synchronized Clock:** Tutti i nodi condividono un clock globale sincronizzato, ossia, $\forall x, y \in \mathcal{E}, clock_x(t) = clock_y(t)$ per ogni istante di tempo t . Questa restrizione semplifica la progettazione dei protocolli, poiché consente ai nodi di coordinare le loro azioni basandosi su un riferimento temporale comune.

Cost & Complexity Measures In generale in un sistema distribuito non ha senso parlare di complessità temporale, poiché non è detto che i nodi condividano un clock globale. Per parlare di complessità temporale è necessario assumere la *Synchronized Clock* e la *Bounded Communication Delay*.

Parleremo spesso di **message complexity**, ovvero una misura che tiene conto del numero di messaggi scambiati durante l'esecuzione di un protocollo. Parleremo quindi di numero di bit scambiati, o di numero di messaggi scambiati.

Capitolo 2

Broadcast

Il primo esempio di comunicazione tra processi è il *broadcast*. Consideriamo un sistema distribuito dove un'entità ha una qualche informazione da condividere con tutte le altre entità. Il processo di inviare un messaggio a tutti gli altri processi è chiamato *broadcast*.

Sia $\mathcal{E}$ l'insieme di tutte le entità nel sistema distribuito e sia $G = (V, E)$ il grafo che rappresenta la rete di comunicazione, dove $V = \mathcal{E}$ e E è l'insieme degli archi che rappresentano le interconnessioni tra i nodi.

Prima di procedere con l'algoritmo di broadcast, è importante assumere alcune restrizioni sul sistema distribuito:

Bidirectional Links (BL): Consideriamo un grafo non diretto, dove ogni arco rappresenta una connessione bidirezionale tra due nodi.

Total Reliability (TR): Assumiamo che i messaggi inviati tra i nodi siano sempre consegnati senza errori o perdite.

Connectivity (CN): Il grafo G è connesso, il che significa che esiste un percorso tra ogni coppia di nodi nel grafo. Questa assunzione è fondamentale per garantire la risoluzione del problema di broadcast, poiché se il grafo non fosse connesso, alcuni nodi non potrebbero ricevere il messaggio.

Unique Initiator (UI): Supponiamo che ci sia un unico nodo che inizia il processo di broadcast, chiamato *initiator*. Questo nodo è responsabile di inviare il messaggio iniziale a tutti gli altri nodi.

Formalmente, il problema di broadcast può essere definito come segue: dato un grafo $G = (V, E)$ e un nodo $s \in V$ che è l'*initiator*, l'obiettivo è garantire che ogni nodo $v \in V$ riceva un messaggio da s .

Definiamo l'insieme $\Sigma = \{\text{initiator}, \text{idle}, \text{informed}\}$ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$.

Definiamo ora le configurazioni del sistema. Una configurazione C è una funzione che associa a ogni nodo $x \in \mathcal{E}$ uno stato in Σ . Inizialmente, la configurazione C_{init} è tale che esiste unico nodo $s \in V$ tale che $C_{init}(s) = \text{initiator}$ e $C_{init}(x) = \text{idle}$ per ogni nodo

$x \in V \setminus \{s\}$. Il numero di configurazioni iniziali possibili è

$$\binom{n}{1} = n,$$

dove n è il numero di nodi nel grafo. Questo perché possiamo scegliere qualsiasi nodo come initiator.

L'unica configurazione finale accettabile è quella in cui ogni nodo $x \in V$ è nello stato **informed**, ovvero

$$C_{final}(x) = \text{informed} \quad \forall x \in V.$$

Quindi, il problema del broadcast si formalizza mediante la quadrupla:

$$\langle \mathcal{R}, \Sigma, C_{init}, C_{final} \rangle,$$

dove $\mathcal{R} = \{\text{BL}, \text{TR}, \text{CN}, \text{UI}\}$ è l'insieme delle restrizioni, Σ è l'insieme degli stati, C_{init} è la configurazione iniziale e C_{final} è la configurazione finale desiderata.

2.1 Algoritmo Flooding

L'idea generale dell'algoritmo di broadcast è quella di utilizzare un approccio di flooding, dove ogni nodo che riceve il messaggio lo inoltra a tutti i suoi vicini. L'algoritmo può essere descritto come segue:

Algorithm 1 Algoritmo di Broadcast (Flooding)

```

if  $C(s) = \text{initiator}$  then
     $C(s) \leftarrow \text{informed}$ 
    for all  $v \in N(s)$  do
        send( $m, v$ )
    end for
end if

if  $C(x) = \text{idle}$  then
     $m \leftarrow \text{receive}()$ 
     $C(x) \leftarrow \text{informed}$ 
    for all  $v \in N(x) \setminus \{\text{sender}\}$  do
        send( $m, v$ )
    end for
end if

if  $C(x) = \text{informed}$  then
    do_nothing()
end if

```

Descritto in questa maniera, l'algoritmo di flooding risolve il task nella maniera corretta, e dimostreremo inoltre che termina in un tempo finito. Tuttavia, è importante notare che questo algoritmo non è efficiente in termini di complessità di messaggi, poiché ogni nodo

potrebbe ricevere lo stesso messaggio più volte da diversi vicini, portando a un numero esponenziale di messaggi inviati.

Osservazione. Se non considerassimo l'insieme degli stati $\Sigma = \{\text{initiator}, \text{idle}\}$ e non avessimo la condizione che un nodo non inoltra il messaggio se è già informato, l'algoritmo di flooding potrebbe non terminare, poiché i nodi continuerebbero a inoltrare il messaggio indefinitamente.

Lemma 2.1.1. *L'algoritmo di flooding termina in un tempo finito. Sia $\text{diam}(G)$ il diametro del grafo G , ovvero la massima distanza tra due nodi qualsiasi nel grafo.*

Esiste un tempo massimo τ tale che tutti i nodi x a distanza h da s ricevano il messaggio m entro il tempo τ .

$$\forall h \leq \text{diam}(G), \exists \tau \in \mathbb{R}^+ : \forall x \in V \mid \text{dist}(x, s) = h \implies C_f(x) = \text{informed} \wedge t(x) \leq \tau.$$

Dimostrazione. Dimostriamo il lemma per induzione su h .

Passo Base $h = 1$ Sfruttando la condizione di Total Reliability, ogni nodo v adiacente a s riceve il messaggio m entro un tempo finito τ_1 . Quindi, per ogni nodo v tale che $\text{dist}(v, s) = 1$, abbiamo $C_f(v) = \text{informed}$ e $t(v) \leq \tau_1$.

Passo Induttivo $h + 1$ Sfruttando la condizione di Connectivity, per ogni nodo $x \in V \setminus \{s\} : d(s, x) = h + 1$.

Questo implica che esiste un shortest path P da $s \rightarrow x$ di lunghezza $h + 1$. Sia y il nodo adiacente a x su questo percorso, quindi $\text{dist}(y, s) = h$. Per ipotesi induttiva, esiste un tempo finito τ_h tale che $C_{final}(y) = \text{informed}$ e $t(y) \leq \tau_h$.

Sfruttando nuovamente la condizione di Total Reliability, il nodo x riceve il messaggio da y entro un tempo finito τ_{h+1} , quindi $C_{final}(x) = \text{informed}$ e $t(x) \leq \tau_{h+1}$.

□